

KV Cache Is Not About Speed

It's About Surviving Inference Costs

David vonThenen
AI/ML Engineer



[@davidvonthenen](#)



David vonThenen

- Agentic Solutions
- Book In Progress
- AI Harness Wizard
- AI/ML Security Hacker
- Part-time GPU Owner

    [@davidvonthenen](https://twitter.com/davidvonthenen)



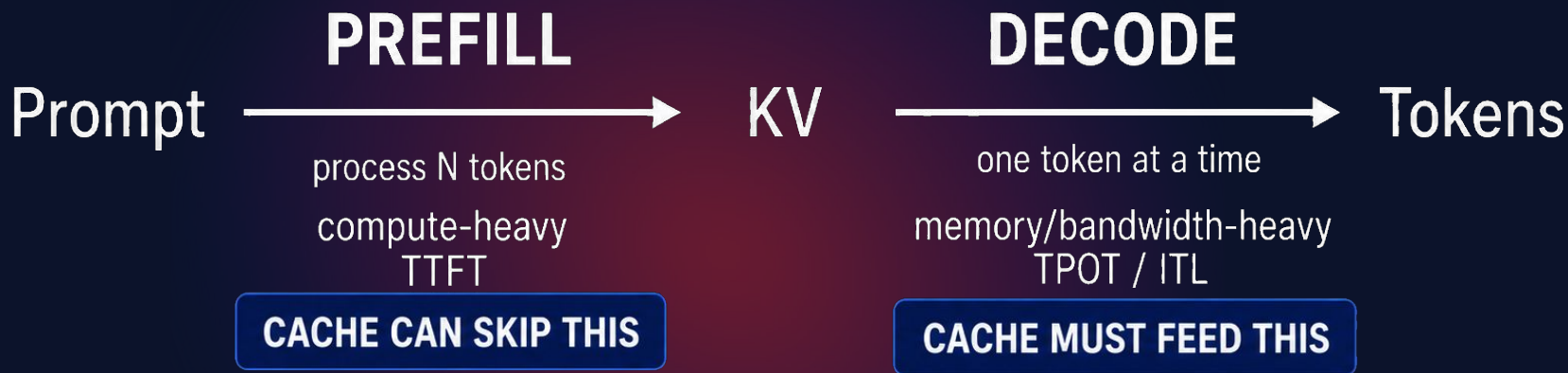
Agenda

- We Love To Talk Speed...
- KV Cache and Inference Costs \$\$\$
- **Demo:** KV Cache, Scale, and Savings
- Q & A

**What If the Expensive Part Isn't Wait
400 Milliseconds Longer?**

**What If the Expensive Part is That We Keep
Renting GPU Cycles to Reproduce Something
We Computed a Few Seconds Ago?**

Inference Has 2 Different Costs



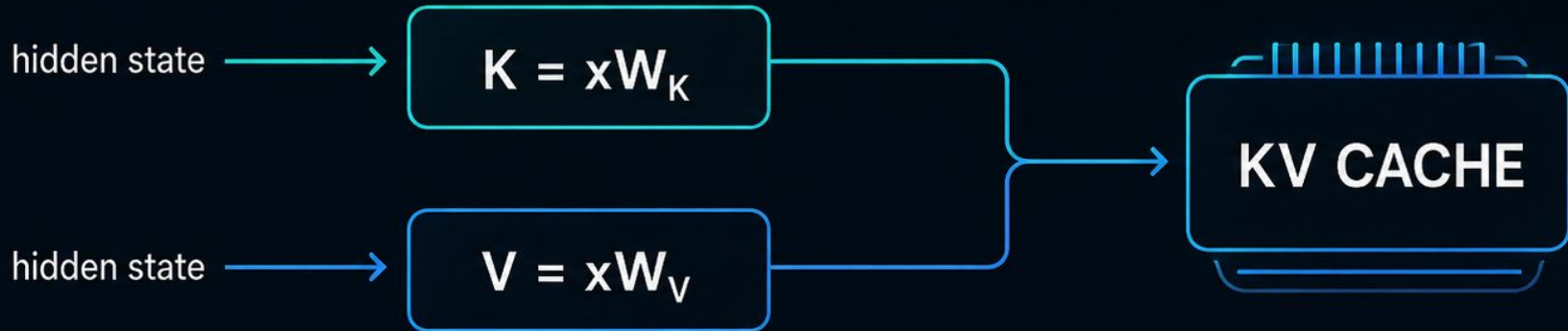
Prefill Ingests:

User Prompt / Question	System Instructions	Irrelevant Data
RAG Documents	MCP/Tool Definitions	"Please..."
Domain Policies	Conversation History	"Thank you"

What Are We Actually Caching?

Image Attribution:
ChatGPT Generated

For every token, at every Transformer layer:



A token's KV state depends on the tokens before it. The same visible token at a different position or after a different prefix may have a different KV representation.

Key = What kind of information do I represent?

Value = What information should I contribute?

Takeaway: Speed is the **Rv_Product**



What users see:

Lower TTFT



What operators see:

Fewer prefills



What finance sees:

More requests / GPU-hour



What the GPU sees:

Less duplicated work

Image Attribution:
ChatGPT Generated

Bottom Line: Reusing a Valid Prefix KV Cache Lets the Inference Engine Skip the Transformer Computation

KV Cache and Inference \$\$\$

One Request Can Eat All

Resources

Let's Do The Math on Qwen 2.5 7B (No KV Cache):

- BF16 Model (dtype) (2 bytes)
- 28 Layers – Dictates Depth of the Model
- 3584 Hidden Size – Dimensional Space ($3584 / 28 = 128$)
- 28 Attention Heads – Focus on Different Relationships
- 4 KV Heads – 7 Atten. Heads Share a Single KV Head ($28 / 4 = 7$)

Doing the Math:

$2 \times 28 \times 4 \times 128 \times 2$ (BF16) = 57,344 bytes/token $\approx$ 56 KB/token

Equation = Key/Vec Vals x Layers x KV Heads x (Hidden Size / Layers) x BF16

Active Context: 8k token = 448MB, 16k = 896MB, 32k = 1.75GB

This is a Single 32k Token Chat!

This 1.75 GB per 32k Token Conversation...

- 8 Users Means Consuming 14 GB of GPU Memory

Remember, This Memory Competes With:

- Batching
- Model Weights
- Runtime/Workspace Memory
- CUDA Graphs and Temporary Buffers
- KV State For Every Other Activate Request

Upshot: Cache Cuts Down On Duplication!



Image Attribution

<https://huggingface.co/Owen/Owen2.5-7B-Instruct>

Economics of a Cache Hit

- Don't Optimize for "Cache Hit Rate"!
- Optimize For:
 - Requests / GPU-second Under SLO
 - Cached Tokens / Request
 - Prefill GPU-ms Avoided
 - Cost / Request
 - TTFT (Remember, Use Cases Like ASR)

How Effective Is Your KV Cache?

90% Cache Hit, But Hits Av. 100 Tokens

40% Cache Hit On 20,000 Token Prefixes = **Transformative!**

t=0: REQUEST 1	t=1: REQUEST 2																																								
"You are an expert in LLM inference optimization. You understand KV caching, paged attention, and memory management. Be concise. How does prefix caching reduce time to first token?"	"You are an expert in LLM inference optimization. You understand KV caching, paged attention, and memory management. Be concise. What is the block size in VLLM?"																																								
(80 tokens → 5 blocks)	(80 tokens → 5 blocks)																																								
HASH	HASH																																								
h0 = 0xA1 h1 = 0xB2 h2 = 0xC3 h3 = 0xD4 h4 = 0xE5	h0, h1, h2 = 0xA1, 0xB2, 0xC3 - same h3 = 0xF6, h4 = 0xG7 - different																																								
CACHE LOOKUP	CACHE LOOKUP																																								
cache: { empty }	cache: { 0xA1-phy_0, 0xB2-phy_1, ... }																																								
all MISS	0xA1-HIT 0xB2-HIT 0xC3-HIT 0xF6-MISS																																								
longest_hit = 0 blocks	longest_hit = 3 blocks (48 tokens)																																								
ALLOCATE & BLOCK TABLE	ALLOCATE & BLOCK TABLE																																								
free: [0,1,2...] → pop 5	free: [5,6,7...] → pop 2																																								
<table><tr><td>Blk0</td><td>Blk1</td><td>Blk2</td><td>Blk3</td><td>Blk4</td></tr><tr><td>phy_0</td><td>phy_1</td><td>phy_2</td><td>phy_3</td><td>phy_4</td></tr><tr><td>ref:1</td><td>ref:1</td><td>ref:1</td><td>ref:1</td><td>ref:1</td></tr><tr><td>new</td><td>new</td><td>new</td><td>new</td><td>new</td></tr></table>	Blk0	Blk1	Blk2	Blk3	Blk4	phy_0	phy_1	phy_2	phy_3	phy_4	ref:1	ref:1	ref:1	ref:1	ref:1	new	new	new	new	new	<table><tr><td>Blk0</td><td>Blk1</td><td>Blk2</td><td>Blk3</td><td>Blk4</td></tr><tr><td>phy_0</td><td>phy_1</td><td>phy_2</td><td>phy_5</td><td>phy_6</td></tr><tr><td>ref:2</td><td>ref:2</td><td>ref:2</td><td>ref:1</td><td>ref:1</td></tr><tr><td>reuse</td><td>reuse</td><td>reuse</td><td>new</td><td>new</td></tr></table>	Blk0	Blk1	Blk2	Blk3	Blk4	phy_0	phy_1	phy_2	phy_5	phy_6	ref:2	ref:2	ref:2	ref:1	ref:1	reuse	reuse	reuse	new	new
Blk0	Blk1	Blk2	Blk3	Blk4																																					
phy_0	phy_1	phy_2	phy_3	phy_4																																					
ref:1	ref:1	ref:1	ref:1	ref:1																																					
new	new	new	new	new																																					
Blk0	Blk1	Blk2	Blk3	Blk4																																					
phy_0	phy_1	phy_2	phy_5	phy_6																																					
ref:2	ref:2	ref:2	ref:1	ref:1																																					
reuse	reuse	reuse	new	new																																					
PREFILL	PREFILL																																								
COMPUTE COMPUTE COMPUTE COMPUTE COMPUTE	SKIP SKIP SKIP COMPUTE COMPUTE																																								
total: 80 tokens	total: 32 tokens (saved 48!)																																								

Image Attribution

How prompt caching works - Paged Attention and Automatic Prefix Caching plus practical tips

<https://sankalp.bearblog.dev/how-prompt-caching-works/>

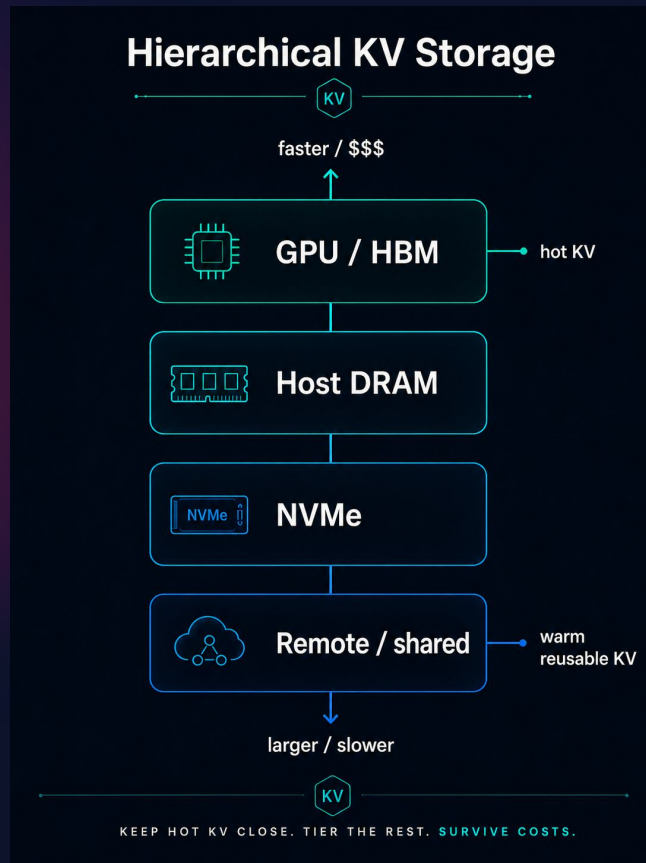
The Cache \$\$\$ Hierarchy

A "Memory Stack" Centric View:

- Leverages GPU as L1
 - SGLang HiCache
- Host DRAM as L2
- Distributed Storage as L3
 - NIXL Plugins (NetApp!)

Distributed Unified Memory Layer

- NVIDIA Dynamo
- Mooncake

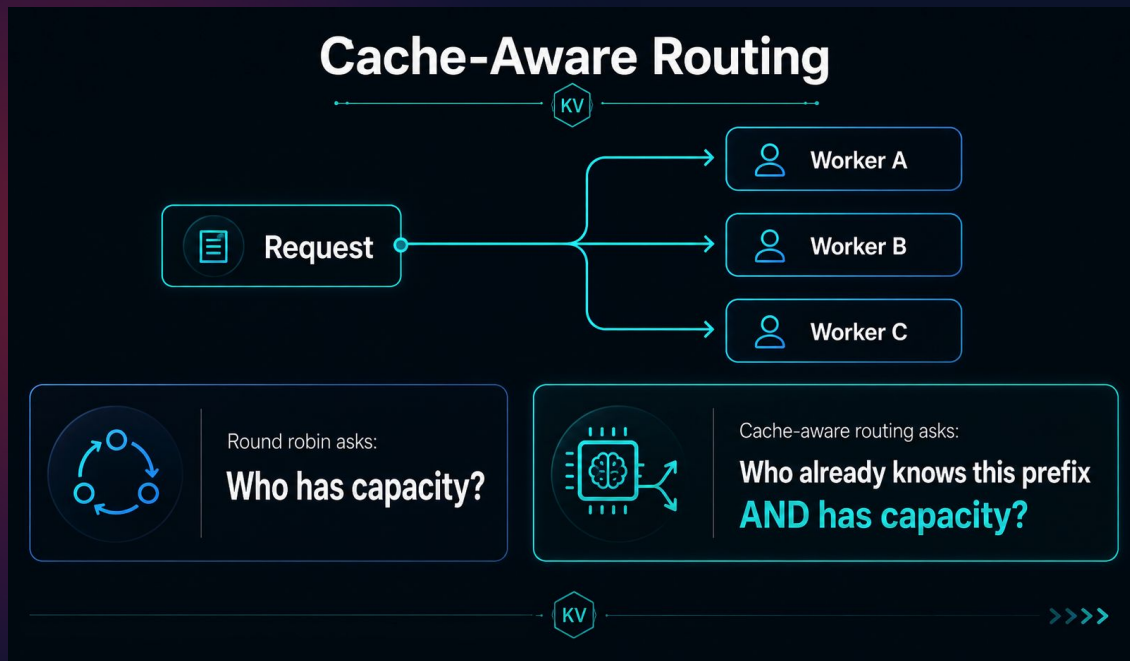


Routing **IS** Memory Management

With Multi-node
Inference...

Combine Predicted
Reusable KV State
With Live Worker
Load

Intelligent Routing is
Global-like Cache!



KV Cache Landscape & Solutions

Technology	Architectural Role	Strong Fit
vLLM	Inference engine; PagedAttention + exact-prefix APC	High-throughput serving and GPU-local prefix reuse
SGLang	Inference/runtime; RadixAttention + hierarchical HiCache	Shared-prefix programs, agents, cache-aware serving
TensorRT-LLM	NVIDIA-optimized inference stack; block reuse/offload	NVIDIA-specific deployments needing tightly optimized serving
LMCache	External/shared KV data layer	Cross-process, cross-engine and tiered KV persistence
Mooncake / Dynamo	Cluster-level scheduling/disaggregation	Large fleets where routing and KV movement must be coordinated

Published Evidence

System / Evidence	Published Result	What It Demonstrates
vLLM / PagedAttention	2-4x throughput at comparable latency in its evaluation	<u>KV memory efficiency translates into serving capacity.</u>
LMCache + vLLM	Up to 15x throughput on evaluated multi-round QA/document-analysis workloads	<u>Reuse beyond GPU-local KV can remove large amounts of repeated prefill.</u>
SGLang cache-aware router	Hit rate 20% → 75% , throughput 82,665 → 158,596 tok/s in its balanced shared-prefix benchmark	<u>Routing can determine whether cached state is economically useful.</u>
Mooncake / Kimi	75% more requests on reported real workloads	<u>KV-centric cluster architecture can increase fleet-level capacity.</u>

Call to Action: Design for Cache

Hits

Prefix Caching Rewards Prompt Stability By:

- Put Stable Content First
- Put Request-Specific Content Last
- Canonicalize Prompts and Templates
- Namespace Cache by Identity
- Route for Cache Locality + Load
- Tier KV by Reuse Probability
- Token-Weighted Hits, Not Request Hits
- Test Cold / Warm / Mutation Paths



Demo Architecture

Image Attribution:
ChatGPT Generated

Cross-instance KV-cache reuse with vLLM + LMCache on one GH200 node

GH200 Instance

ARM CPU + 1x H100 96 GB

Shared H100 GPU

Benchmark Driver

multi_round_qa.py

- OpenAI-compatible requests
- concurrent users / rounds / QPS
- CSV output + summary

Results

- TTFT
- throughput
- cache-hit metrics
- summary CSV



HAProxy

http://127.0.0.1:7999
round robin



vLLM Instance A

Qwen3-8B
http://127.0.0.1:8000
max-model-len 16384
LMCacheMPCConnector



vLLM Instance B

Qwen3-8B
http://127.0.0.1:8001
max-model-len 16384
LMCacheMPCConnector

read / write

shared KV-cache plane

read / write



LMCache Server

- ZMQ: 127.0.0.1:5555
- HTTP / metrics: 127.0.0.1:8080
- L1 CPU memory cache
- 8-16 GB
- Eviction: LRU



Goal: demonstrate prefix reuse across two vLLM processes

Demo

<https://youtu.be/pmai7flae1s>

Benchmark Results

Metric	Result
Request per GPU-sec	0.5915
Average Prompt	9,391 tokens
Direct Cached Token	12,288 tokens
Est Cacheable Token Per-Request	~8,913
Cacheable Prompt Fract	~94.9%
Est Prefill GPU-ms Avoid Per-Request	~176-194ms
Aggressive Full-Hit Case	~7 nodes

Metric	Result
Cold TTFT	330.8ms
Cache-hit TTFT	63.7ms
Direct TTFT Speedup	5.2x
Ave Multi-Round TTFT	204.5ms
P96 TTFT	224.8ms
H100 Equiv Cap Avoided	~10-11.5% per GPU
Theoretical Nodes per Freed H100	~9-10
GH200 cost per request at tested QPS	\$0.00108

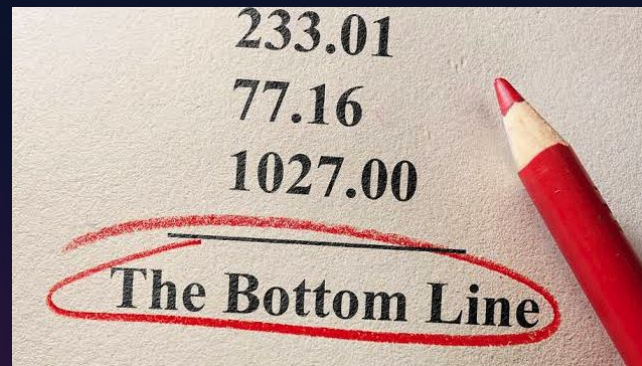
The Bottom Line

The "WOW" Factor:

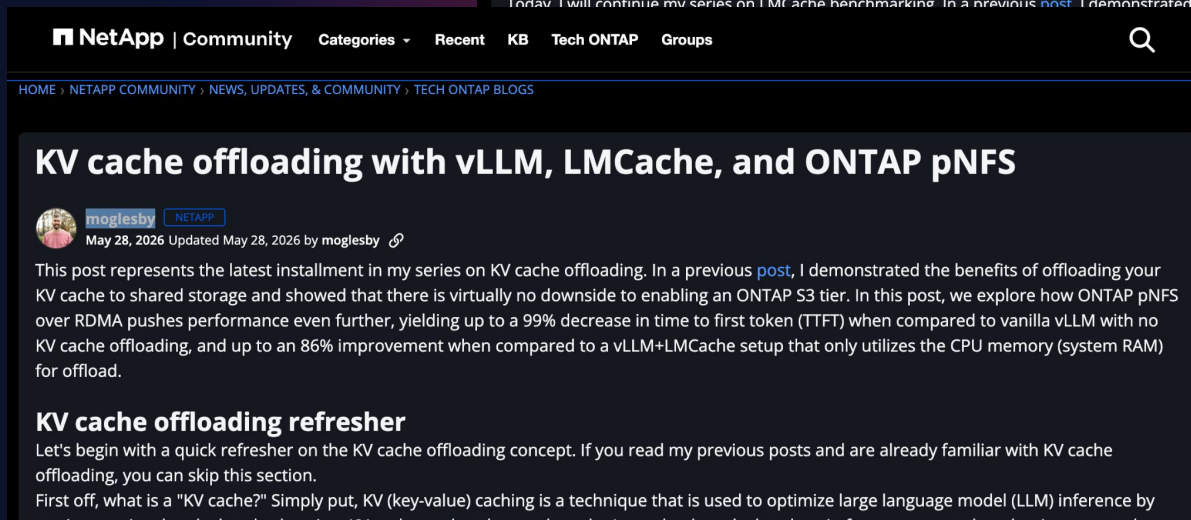
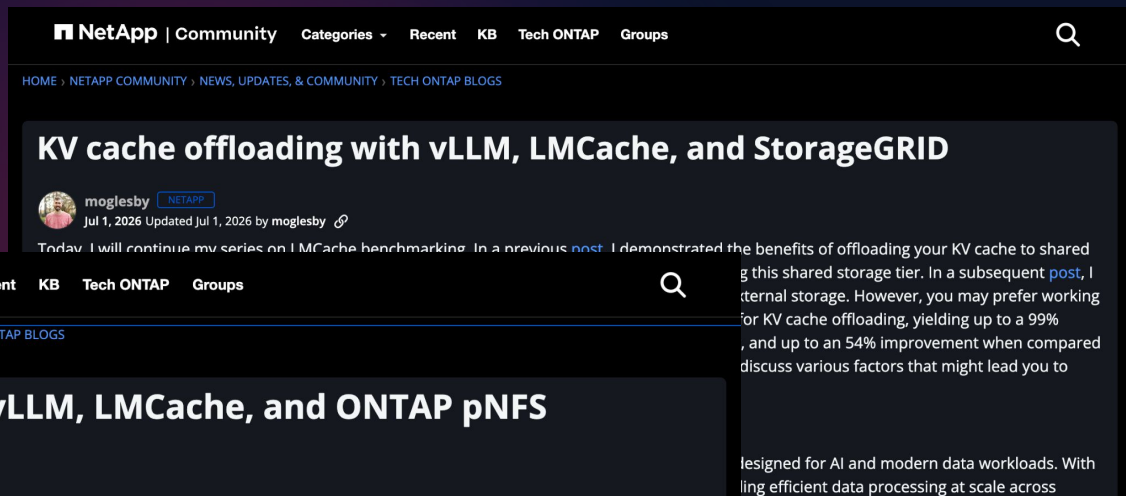
- 12k Cross Instance Cache Hit
 - Cuts TTFT from 331ms to 64ms
- Multi-round Workload
 - Cuts ~10-11% of H100-Equiv. Redundant Prefill Compute
- Scaling Inference Workloads
 - At 9-10 GPUs, The Accumulated Savings is an H100
 - Effectively Reclaim a GPU for Another Workload

Quantized Differences – BF16 versus FP8:

- Less Mem. Consumed By Weights, More for Active Inference
- LMCACHE: Less Redundant Prefill Compute = Less GPU Work / Repeated Context



More Boom for Buck!



Resources

Resources

SCAN THE QR CODE FOR ALL RESOURCES!!

github.com/davidvonthenen/2026-wearedevelopers-na

NetApp Blog Posts on KV Cache:

- [KV Cache with ONTAP pNFS, StorageGRID](#)
- [Blog](#), [Blog](#), [Blog](#), [Blog](#), and more [Blog](#)!

Launching Instacluster Labs

github.com/instacluster/labs

Launching Instacademy

instacluster.com/instacademy-courses



Thank You!



David vonThenen
Senior AI/ML Engineer



@davidvonthenen

